SQL Performance Tuning – Notes

1. Introduction

SQL performance tuning refers to the process of optimizing database queries and structures to improve the speed and efficiency of data retrieval and manipulation. When queries are not optimized, they can consume excessive system resources such as CPU, memory, and disk I/O. This leads to slow database performance, longer response times, and sometimes system downtime.

Effective SQL performance tuning ensures that queries execute quickly, server load is minimized, and database resources are used efficiently. It plays a crucial role in maintaining stable and scalable database systems, especially in environments with large datasets and multiple concurrent users.

2. Factors Affecting SQL Query Performance

1. Table Size
Large tables containing millions of records can slow down query execution. Queries that scan large portions of a table require more processing time and system resources.

2. Joins
Joining multiple tables increases the complexity of query execution. Complex joins may cause large intermediate result sets, increasing processing time.

3. Aggregations
Operations such as SUM, AVG, COUNT, and GROUP BY require additional computation, especially when applied to large datasets.

4. Concurrency
When many users run queries simultaneously, the database server must handle multiple requests at once. This can lead to contention for resources and slower query execution.

5. Indexes
Indexes improve data retrieval speed by allowing the database to locate rows quickly. However, poorly designed indexes can reduce performance, especially during insert or update operations.

3. Methods to Identify Slow SQL Queries

3.1 Execution Plan

An execution plan shows how the database engine executes a query. It provides information about operations such as table scans, index usage, and join methods.

Steps to generate an execution plan in SQL Server Management Studio:

1. Open a new Database Engine Query window.
2. Write the SQL query.
3. Enable the “Include Actual Execution Plan” option.
4. Execute the query.
5. Review the execution plan displayed in the results pane.

Execution plans help identify inefficient operations such as full table scans or missing indexes.

3.2 Monitoring Resource Usage

Database performance depends heavily on system resources including CPU, memory, and disk activity. Tools like Windows Performance Monitor allow administrators to observe these metrics and detect performance bottlenecks. Monitoring both SQL Server counters and operating system metrics helps identify correlations between resource usage and query performance issues.

3.3 Dynamic Management Views (DMVs)

SQL Server provides Dynamic Management Views that allow administrators to analyze query execution statistics and identify slow-running queries.

Example DMV:
sys.dm_exec_query_stats

These views provide information such as:

- Execution count
- Query execution time
- Resource consumption
- Query plans

Using DMVs helps identify inefficient queries that need optimization.

4. SQL Query Optimization Techniques

4.1 Avoid Using SELECT *

Using SELECT * retrieves all columns from a table, even when only a few are required. This increases data transfer and processing time.

Inefficient:

```
SELECT * FROM GeeksTable;
```

Efficient:

```
SELECT FirstName, LastName, Address, City, State, Zip

FROM GeeksTable;
```

4.2 Avoid SELECT DISTINCT When Possible

SELECT DISTINCT removes duplicate records but requires additional processing. Instead, design queries so that duplicates are naturally avoided.

Inefficient:

```
SELECT DISTINCT FirstName, LastName, State
```

```
FROM GeeksTable;
```

Efficient:

```
SELECT FirstName, LastName, State
FROM GeeksTable
WHERE State IS NOT NULL;
```

4.3 Use INNER JOIN Instead of WHERE for Joins

Using explicit join syntax improves readability and often leads to better query optimization by the database engine.

Inefficient:

```
SELECT GFG1.CustomerID, GFG1.Name, GFG1.LastSaleDate
FROM GFG1, GFG2
WHERE GFG1.CustomerID = GFG2.CustomerID;
```

Efficient:

```
SELECT GFG1.CustomerID, GFG1.Name, GFG1.LastSaleDate
FROM GFG1
INNER JOIN GFG2
ON GFG1.CustomerID = GFG2.CustomerID;
```

4.4 Use WHERE Instead of HAVING

The WHERE clause filters records before grouping, while HAVING filters results after aggregation. Filtering earlier reduces the amount of data processed.

Inefficient:

```
SELECT CustomerID, Name
FROM GFG1
GROUP BY CustomerID, Name
HAVING LastSaleDate BETWEEN '2019-01-01' AND '2019-12-31';
```

Efficient:

```
SELECT CustomerID, Name
FROM GFG1
WHERE LastSaleDate BETWEEN '2019-01-01' AND '2019-12-31'
GROUP BY CustomerID, Name;
```

4.5 Limit Wildcards in Search Conditions

Using % at the beginning of a pattern prevents the use of indexes and slows down searches.

Inefficient:

```
SELECT City FROM GeekTable

WHERE City LIKE '%No%';
```

Efficient:

```
SELECT City FROM GeekTable

WHERE City LIKE 'No%';
```

4.6 Use LIMIT for Testing Queries

The LIMIT clause restricts the number of returned rows. It is useful when testing queries on large tables to avoid unnecessary processing.

Example:

```
SELECT CustomerID, Name

FROM Customers

LIMIT 10;
```

4.7 Run Heavy Queries During Off-Peak Hours

Running large analytical queries during periods of low system usage reduces the impact on other users. This is often scheduled overnight or during maintenance windows.

5. Index Tuning

Index tuning involves selecting appropriate columns to index so that query execution becomes faster.

Important considerations when creating indexes:

- **Short Indexes:** Smaller indexes reduce disk usage and allow faster comparisons.
- **High Selectivity:** Columns with many unique values make better indexes.
- **Clustered Indexes:** Store table data in sorted order based on the indexed column.
- **Static Columns:** Columns that rarely change are good candidates for clustered indexes.

While indexes improve read performance, they may slow down insert and update operations because indexes must also be updated.

6. SQL Performance Tuning Tools

Several tools help monitor database performance and identify slow queries.

SQL Sentry (SolarWinds)
Provides real-time monitoring, performance analysis, and query optimization insights.

SQL Profiler (Microsoft)
Captures and analyzes SQL Server events, helping identify slow queries and performance issues.

SQL Index Manager (Red Gate)
Assists in analyzing and optimizing database indexes.

SQL Diagnostic Manager (IDERA)
Provides comprehensive monitoring of SQL Server performance and resource usage.

These tools help database administrators detect inefficiencies and improve query performance.

Database Normalization – Notes

1. Introduction

Database normalization is a systematic process used to organize data in a database to reduce redundancy and improve data integrity. The main objective is to divide large tables into smaller related tables while maintaining logical relationships between them.

Normalization ensures that data is stored efficiently and prevents anomalies that can occur during data insertion, deletion, and updates.

2. Problems Without Normalization

Insertion Anomaly

Occurs when new data cannot be inserted because other required data is missing.

Example:
If a department table requires employee details, a department without employees cannot be stored.

Update Anomaly

Occurs when the same data appears in multiple rows and must be updated in each location.

Example:
If a department location changes, all rows containing that department must be updated.

Deletion Anomaly

Occurs when deleting one record unintentionally removes additional information.

Example:
If the last employee in a department is deleted, the department information may also be lost.

Data Redundancy

The same data is stored multiple times, increasing storage requirements and creating consistency problems.

3. Need for Normalization

Normalization helps to:

- Reduce duplicate data
- Improve data consistency
- Simplify database maintenance

- Prevent insertion, update, and deletion anomalies
- Improve database structure and flexibility

4. Features of Database Normalization

Elimination of Data Redundancy
Normalization removes repeated data and ensures each piece of information is stored only once.

Data Consistency
Ensures that all references to the same data remain consistent across the database.

Simplified Data Management
Breaking large tables into smaller ones makes data easier to manage and maintain.

Improved Database Design
A normalized structure improves scalability and flexibility.

Avoidance of Update Anomalies
Changes to data only need to be made in one place, preventing inconsistencies.

Standardized Data Organization
Data is structured logically with clear relationships between tables.

5. Normal Forms

Normalization is achieved through different levels called normal forms.

First Normal Form (1NF)

A table is in 1NF if:

- Each column contains atomic values.
- There are no repeating groups or multi-valued attributes.

Second Normal Form (2NF)

A table is in 2NF if:

- It is already in 1NF.
- Every non-key attribute is fully dependent on the primary key.

This eliminates partial dependency.

Third Normal Form (3NF)

A table is in 3NF if:

- It is already in 2NF.

- There are no transitive dependencies between non-key attributes.

This ensures non-key attributes depend only on the primary key.

Boyce-Codd Normal Form (BCNF)

BCNF is a stronger version of 3NF.

Conditions:

- The table must already be in 3NF.
- For every functional dependency X → Y, X must be a super key.

Fourth Normal Form (4NF)

A table is in 4NF if:

- It satisfies BCNF.
- It has no multi-valued dependencies.

Fifth Normal Form (5NF)

A table is in 5NF if:

- It is already in 4NF.
- It cannot be further decomposed without losing information.

This form deals with complex join dependencies.

Conclusion

SQL performance tuning and database normalization are fundamental techniques for building efficient database systems. Performance tuning focuses on improving query execution and resource utilization, while normalization focuses on organizing data structures to eliminate redundancy and maintain consistency. Together, they ensure reliable, scalable, and high-performing database environments.